SMARTCAMPUS AI INTEGRATION

Technical Specification & Implementation Guide — v1.0 · March 2026
Confidential · Internal Documentation

Audience: Development Team · Product Team · School Administrators
Related Modules: Student Portal, Teacher Portal, Analytics, Reports, Communication

---

1. Overview

This specification outlines how to integrate AI tools into SmartCampus using a **hybrid approach**:

- **Students:** Free access via direct links or embedded widgets (GPT-3.5, Perplexity basic tier).
- **Teachers/Admins:** Paid API integration (OpenAI GPT-3.5 / GPT-4, Perplexity Pro) for advanced automation such as grading, performance summaries, and analytics.

Goals:

1. Provide AI-assisted learning for students at zero cost.
2. Enable automated, high-value admin/teacher tasks.
3. Maintain security, privacy, and cost control.

---

2. System Architecture

2.1 Layers

Layer	Description
Frontend (Student)	Dashboard buttons or embedded AI web widgets (iframe / new tab)

Layer	Description
	pointing to free AI services.
Frontend (Teacher/Admin)	SmartCampus portal integrates backend API calls to AI services (GPT-3.5/GPT-4).
Backend	Handles authentication, role-based routing, API calls, data preprocessing, and result rendering.
AI Services	External APIs (OpenAI, Perplexity) process queries and return responses.

2.2 Hybrid Flow Diagram

```
[Student Portal] ----> [Free AI Widget / Link] ----> [External AI Service
(GPT-3.5)]
[Teacher/Admin Portal] ----> [SmartCampus Backend] ----> [AI API (GPT-4/GPT-
3.5)] ----> [Processed Results] ----> [Dashboard]
```

---

3. Functional Specification

3.1 Student Access (Free Tier)

Features:

- “Ask AI” button on dashboard.
- Opens GPT-3.5 chat in new tab OR embeds iframe (if allowed).
- Queries **do not incur API costs**.
- Responses **not stored in SmartCampus**, optional logging for analytics.

Implementation Notes:

- Frontend: HTML button / embedded iframe.
- Backend: None required; no API key exposure.
- Security: Ensure link opens in new tab or sandboxed iframe.

3.2 Teacher/Admin Access (Paid API)

Features:

- Summarize student grades & attendance.
- Generate class performance reports.
- Auto-generate feedback for assignments/quizzes.
- Predictive analytics (at-risk students, trend forecasting).

Implementation Notes:

- Backend server handles API calls securely.
- API keys stored in server environment variables.
- Input data sanitized & anonymized if required for privacy.
- Role-based access ensures only authorized staff access AI features.

---

4. Technical Specification

4.1 API Integration

OpenAI Example (Python / Node.js backend)

```
import openai
openai.api_key = "YOUR_OPENAI_API_KEY"

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are an educational assistant."},
        {"role": "user", "content": "Summarize student grades for Class 8A
this term."}
    ],
    max_tokens=500
)
print(response.choices[0].message['content'])
```

- **Model options:** GPT-3.5 (low cost), GPT-4 (advanced).
- **Rate limits:** Monitor OpenAI usage dashboard to manage cost.
- **Output:** JSON returned to backend → render in portal.

4.2 Role-Based Access

- Students → free AI link/widget only.
- Teachers → access via backend API integration.
- Admins → full AI dashboard with analytics and report generation.
- Use RBAC to enforce permissions at API call and module level.

---

5. Security & Privacy

- **HTTPS:** All backend API calls must be over TLS.
- **API Key Security:** Store in server env variables; never in client code.
- **Data Anonymization:** Remove personally identifiable info when sending queries externally.
- **Audit Logs:** Track who made API calls and timestamp for accountability.
- **Compliance:** Align with Kenya Data Protection Act and GDPR-equivalent practices.

---

6. Scalability & Cost Management

- **Student queries:** No cost (free AI).
- **Teacher/Admin queries:** Paid API; implement daily caps to control cost.
- **Monitoring:** Track token usage per teacher/admin; estimate monthly fees.
- **Hybrid Optimization:** Start with free access for students, gradually expand API-based features for high-value tasks.

---

7. User Experience

User Role	Access	Flow
Student	Free AI link / iframe	Click → AI opens → ask questions → read answers
Teacher	API integration	Select task → backend processes → AI summarizes → result shown in dashboard
Admin	API integration	Dashboard → select reports/analytics → AI generates summaries → export/download

- Students are **never redirected to paid API** to avoid cost.
- Teachers/admins **always go through backend** to prevent exposing API keys.
- Dashboard **filters modules** based on role.

---

8. Implementation Steps

1. **Sign Up for API Keys**
   - OpenAI: GPT-3.5 / GPT-4
   - Perplexity Pro (optional)
2. **Frontend Setup**
   - Student portal: embed “Ask AI” button or iframe.
   - Teacher/admin portal: buttons for AI-assisted tasks.
3. **Backend Setup**
   - API endpoints for AI tasks (summaries, analytics, grading).
   - Sanitize and format data before sending to AI.
   - Return structured JSON for dashboard rendering.
4. **RBAC Configuration**
   - Assign roles → map modules & AI access.
   - Implement auto-routing based on role.
5. **Security & Compliance**
   - Store keys securely, enable HTTPS, anonymize sensitive data.
   - Add logging for audit purposes.
6. **Testing**
   - Verify students can access free AI.
   - Verify teachers/admins receive accurate AI results.
   - Monitor token usage and cost.
7. **Deployment & Monitoring**
   - Roll out incrementally.
   - Track API usage and student engagement.
   - Adjust quotas to manage budget.

---

9. Maintenance & Upgrades

- Update AI model versions as they are released.
- Adjust token limits based on usage trends.
- Add new automated workflows gradually (e.g., lesson plan generation, predictive student alerts).
- Monitor privacy compliance and adjust logging/anonymization as regulations evolve.